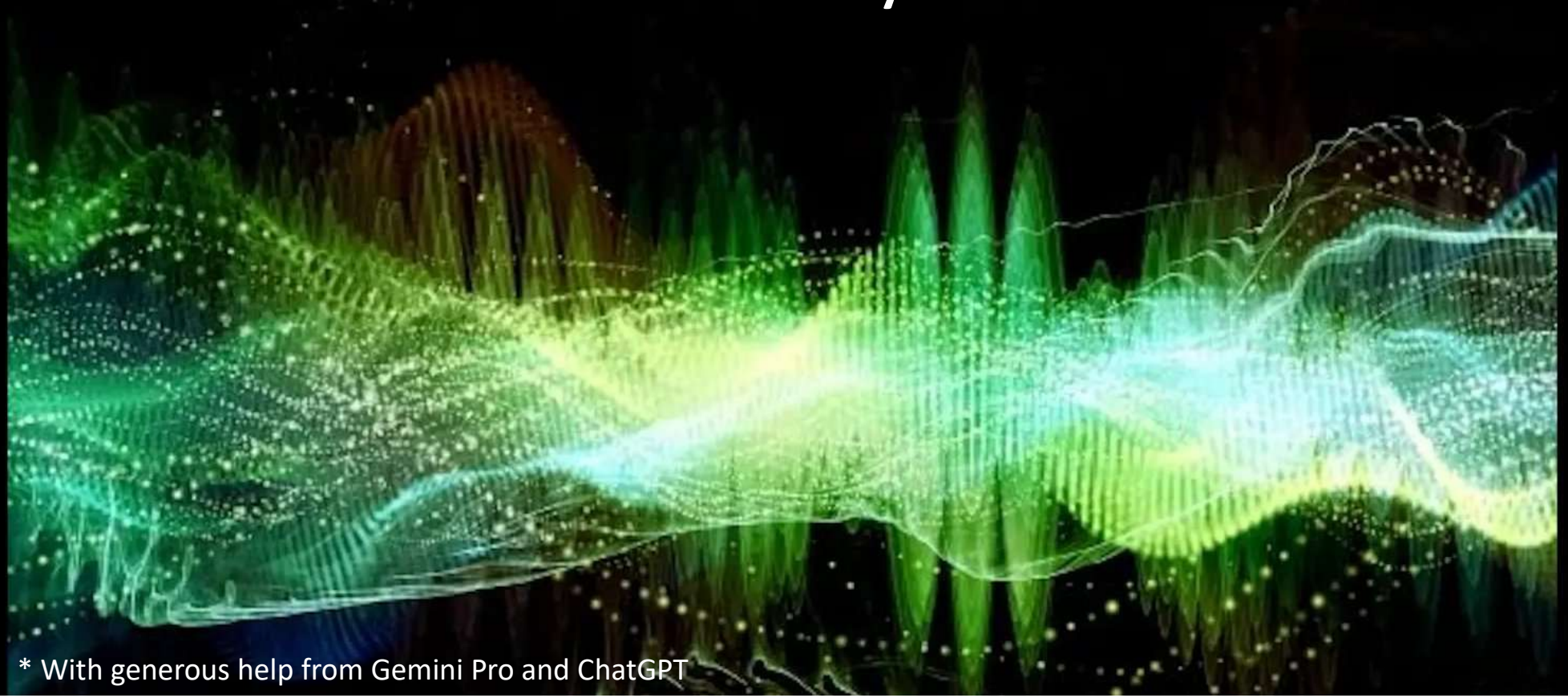
ECEN 4/5632 Theory and Application of Digital Filters
Lecture 11, 10-01-26, 10-06-26, 10-13-26
Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

WEEK 6 ● LECTURE 11

The DFT and Fast Convolution

From frequency samples to efficient streaming FIR filtering

Oppenheim & Schafer • DFT / FFT / STFT | Raschka • CNN architecture and feature pipelines

Learning objectives

ROADMAP

0:00–0:03

- Interpret the N-point DFT as samples of the DTFT of a finite record.
- Distinguish linear convolution from N-point circular convolution.
- Choose zero-padding so DFT multiplication produces linear convolution.
- Explain and implement overlap-add and overlap-save block filtering.
- Connect FFT block processing to real-time preprocessing before ML inference.

Core question

How can an operation that is naturally defined for a whole sequence be implemented efficiently on an endless stream?

sampling DTFT in frequency domain

OLA OLS

need to preserve memory for filtering

$$H(e^{j\omega}) = \sum_n h[n] e^{-j\omega n}$$

time/resource consuming to compute for real-life audio signals

Need to break-up long inputs into blocks

For freq domain processing we need to take DFT/FFT of blocks

$$\text{DFT } H[k] = H(e^{j\omega})\Big|_{\omega = \frac{2\pi k}{N}}$$

N: block length

DFT review: a finite set of frequency samples

DFT

0:03–0:09

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0,\ldots,N-1$$

DFT

- The DFT is exactly the DTFT of the N-sample finite record evaluated at $\omega_k = 2\pi k/N$.
- The inverse DFT reconstructs those N time samples exactly.
- The DFT assumes an N-periodic extension when we interpret algebra such as convolution.

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N} \qquad k = 0, 1, \ldots, N-1$$

DFT
block length N

Do not confuse

A larger FFT can sample the same finite-record DTFT more densely, but zero-padding does not create new time-domain information.

DTFT → DFT

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-j\omega n}$$ DTFT

Sample in freq domain $\omega \to \frac{2\pi k}{N} \to e^{j 2\pi k/N}$ basis frequencies instead of $e^{j\omega}$

signal $x[n]$ available for $0 \le n < N$

$$\sum_{n=0}^{N-1} x[n]\, e^{-j\frac{2\pi k}{N} \cdot n} = X[k]$$ DFT blocklength N

$0 \le k < N$

inverse DFT

$$\frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j\frac{2\pi k n}{N}} = x[n]$$

periodic in k and n with period N

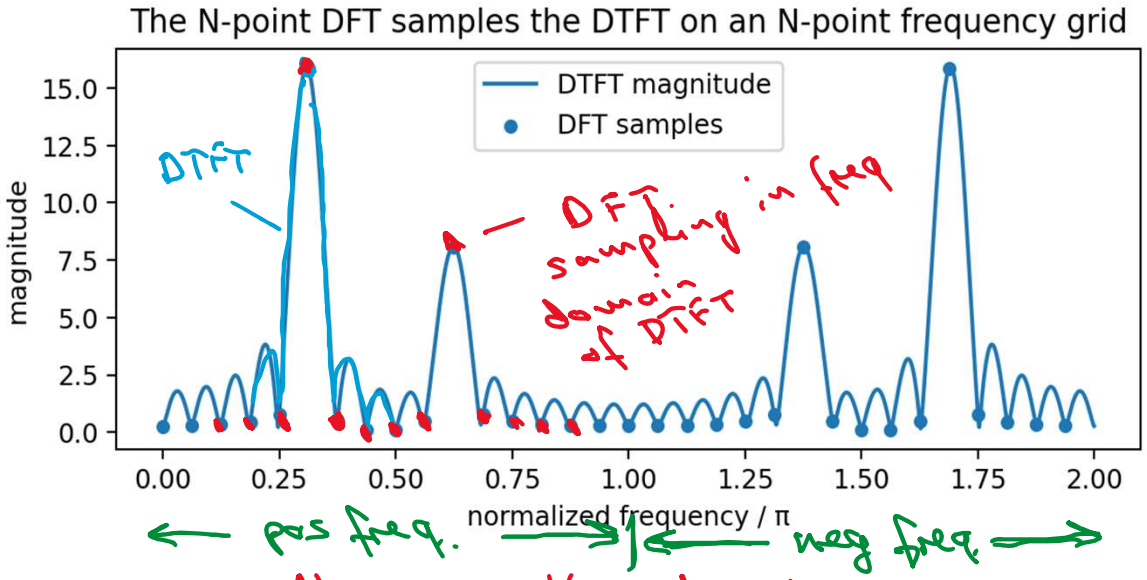
$e^{j2\pi k(n+N)/N} = e^{j\frac{2\pi kn}{N}} \underbrace{e^{j\frac{2\pi kN}{N}}}_{=1}$

k, n indexes have to be taken modulo N because of periodicity

The DFT samples the DTFT

DFT

0:09–0:14



Reading the figure

The smooth curve is the finite record's DTFT. The dots are the 32 DFT samples. An FFT is an algorithm for computing these samples efficiently.

Frequency grid: $\omega_k = 2\pi k/N$

Why the FFT matters computationally

DFT

0:14–0:20

Direct DFT: approximately $O(N^2)$ operations

FFT: approximately $O(N \log_2 N)$ operations

- This difference becomes decisive for long FIR filters and repeated streaming blocks.
- The FFT does not change the transform—it changes the computational algorithm.
- Fast convolution exploits the convolution theorem plus the FFT.

Example

For $N = 4096$, $N^2 \approx 16.8$ million while $N \log_2 N \approx 49$ thousand scale operations (constant factors omitted).

if $N = 1000$

then $O(N^2) = 10^6$

$O(N \log N) = 3000$

FFT is fast way to compute DFT

DFT example $N=4$

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \cdot \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix}$$

$$W_N = e^{-j2\pi/N}$$

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} = \sum_{n=0}^{N-1} x[n] W_N^{kn}$$

DFT Computation

$N=4$ $\qquad W_N = e^{-j2\pi/N}$

$$X_k = x[0]\cdot W_4^{0k} + x[1]W_4^{1k} + x[2]\cdot W_4^{2k} + x[3]W_4^{3k}$$

↑ brute force computation

$$X_k = x[0] + x[1]\cdot W_4^{k} + x[2]\cdot W_2^{k} + x[3]\cdot W_4^{k}\cdot W_2^{k}$$

$$= x[0] + x[2](-1)^k + \left(x[1] + x[3](-1)^k\right) W_4^{k}$$

↑ streamlined computation

$$X_0 = (x[0] + x[2]) + (x[1] + x[3])$$

$$X_1 = (x[0] - x[2]) - j(x[1] - x[3])$$

$$X_2 = (x[0] + x[2]) - (x[1] + x[3])$$

$$X_3 = (x[0] - x[2]) + j(x[1] - x[3])$$

$W_4^0 = 1$

$W_4^2 = e^{-j2\pi\cdot 2/4}$

$= e^{-j2\pi/2}$

$= W_2^1$

$W_4^1 = e^{-j2\pi/4}$

$= e^{-j\pi/2} = -j$

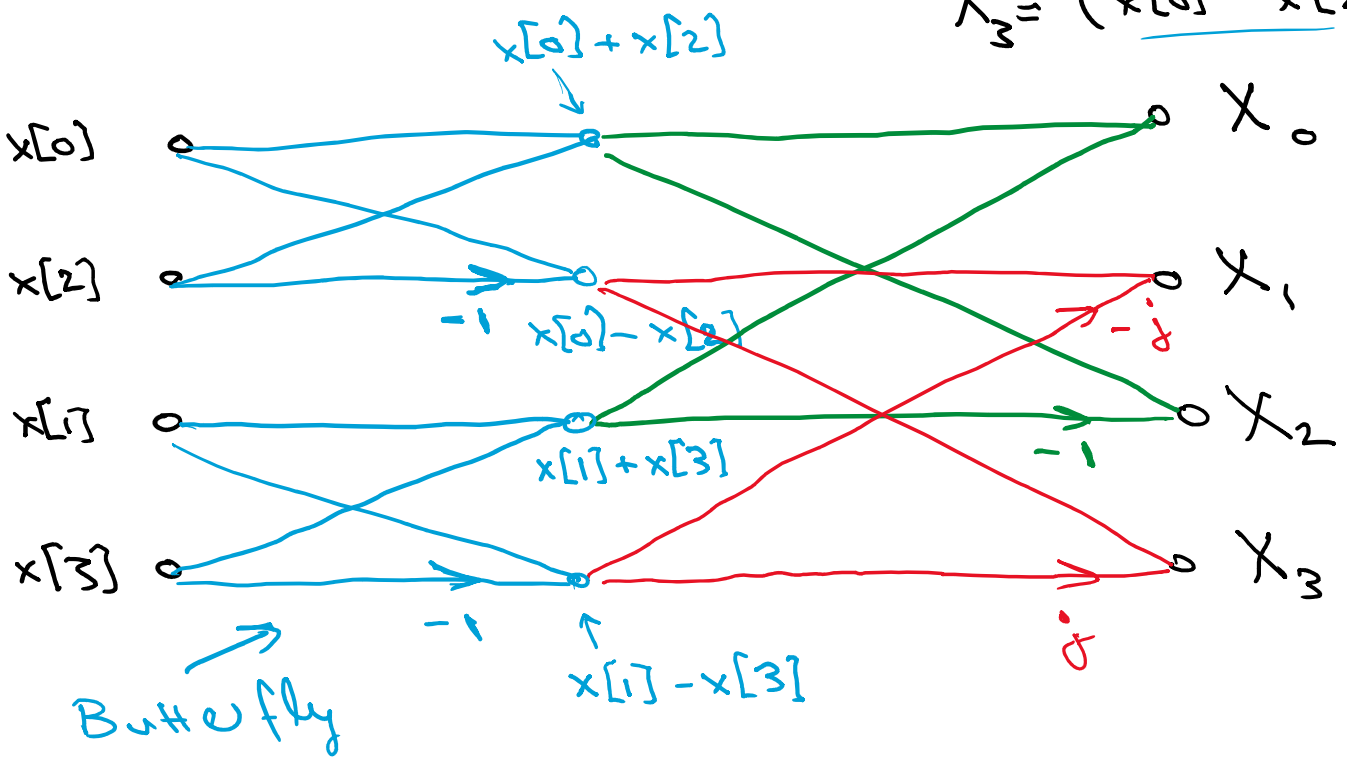
$W_4^3 = W_4^1 \cdot W_4^2$

$= W_4^1 \cdot W_2^1$

DFT Computation

Flowgraph of streamlined computation

$$X_0 = (x[0] + x[2]) + (x[1] + x[3])$$
$$X_1 = (x[0] - x[2]) - j(x[1] - x[3])$$
$$X_2 = (x[0] + x[2]) - (x[1] + x[3])$$
$$X_3 = (x[0] - x[2]) + j(x[1] - x[3])$$



Key to FFT (Fast Fourier Transform)

Linear convolution versus circular convolution

CONVOLUTION

0:20–0:27

Linear: $y[n] = \sum_m x[m]\, h[n-m]$

N-point circular: $y_N[n] = \sum_{m=0}^{N-1} x[m]\, h[(n-m) \bmod N]$

Key issue

If N is too small, the tail of the linear convolution wraps onto the beginning.

- Linear convolution has length Lx + Lh − 1 for finite sequences.
- Circular convolution wraps samples modulo N.
- DFT multiplication corresponds to circular convolution of the N-point sequences.

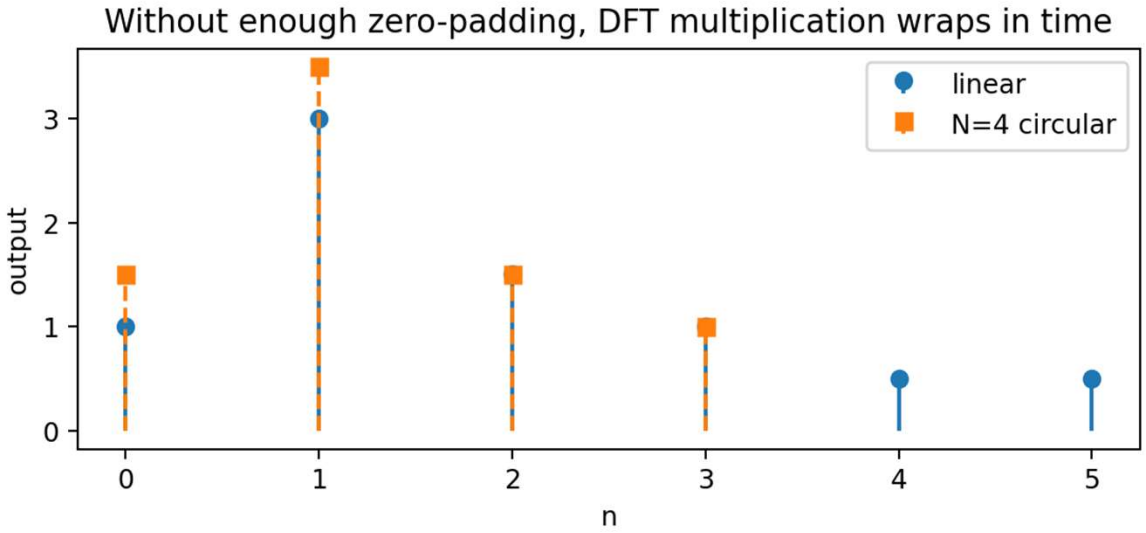
Convolution using FFT – Multiply – inverse FFT is faster than $\sum_k h[k]\, x[n-k]$ for long enough h[k]

Catch: FFT computes circular convolution because of periodicity in n, k different from linear convolution

Wrap-around is the error

CONVOLUTION

0:27–0:32



What happened?

The N=4 circular result folds the samples that belong at n ≥ 4 back into n = 0,1,… .

Circular is not "wrong"—it is a different operation.

One strategy is to make blocklength $N \geq L_h + L_x - 1$

length of $h[n]$ — length of $x[n]$

Zero-padding makes DFT convolution linear

CONVOLUTION

Choose an FFT length satisfying

$$\mathbf{NFFT \geq Lx + Lh - 1}$$

- Pad both x[n] and h[n] to NFFT. ← pad with zeros to length NFFT
- Multiply their NFFT-point DFTs.
- Take the inverse DFT.
- The first Lx + Lh − 1 samples equal the linear convolution.

Example

Lx = 1000 and Lh = 129 require at least 1128 points. In practice, 2048 may be chosen for FFT efficiency.

$2^2 3^2 5^2 7^2 = 44100$ as alternative of 48000

Concept check: padding requirement

CONCEPT CHECK

0:38–0:40

x[n] has length 80 and h[n] has length 33. What is the minimum FFT size for one-shot FFT-based linear convolution?

$L_x = 80 \quad L_h = 33 \qquad 80 + 32 = 112 \Rightarrow N = 128 = 2^7$

Concept check: padding requirement

CONCEPT CHECK

0:38–0:40

x[n] has length 80 and h[n] has length 33. What is the minimum FFT size for one-shot FFT-based linear convolution?

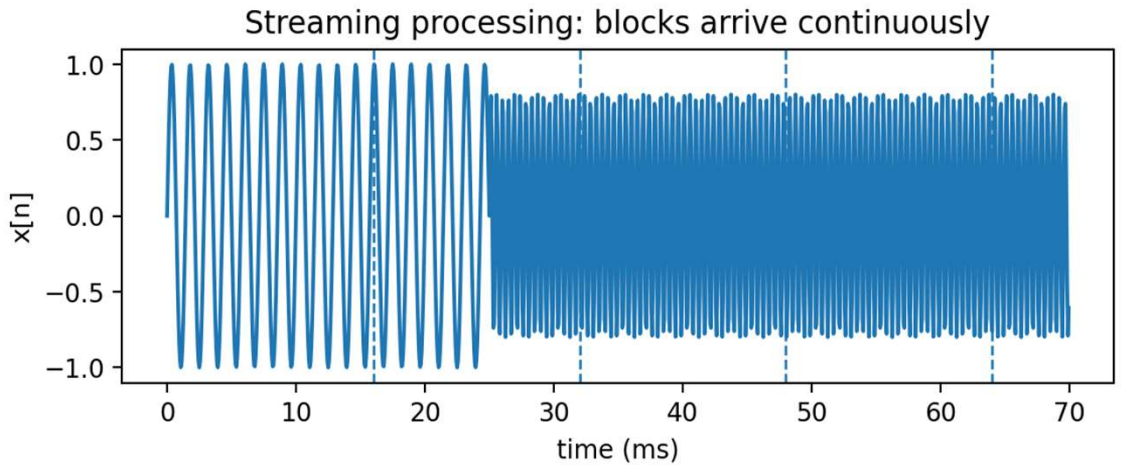
Good answer

NFFT ≥ 80 + 33 − 1 = 112. Any NFFT of at least 112 works mathematically; an implementation may choose 128.

Why block processing is needed

STREAMING

0:40–0:46



- A microphone, sensor, or radio stream does not end so that we can take one giant FFT.
- We process finite blocks as they arrive.
- Each block must still reproduce the same linear FIR filtering result.
- Block length affects throughput, latency, memory, and FFT efficiency.

Filtering things in blocks : How do you keep continuity/memory

ML connection

Real-time inference often begins with streaming DSP: filtering, resampling, denoising, framing, and feature extraction.

For filtering of blocks of x[n] signals, overlap between blocks is needed

Overlap-Add: filter independent input blocks

OVERLAP-ADD

0:46–0:54

1. Split x[n] into non-overlapping blocks of B samples.
2. Zero-pad each block and h[n] so NFFT ≥ B + Lh − 1.
3. FFT → multiply by H[k] → IFFT.
4. Add the overlapping output tails.

OLA overlap-add

OLS overlap-save

block size

length of filter

FFT → $X_i[k]$ · $H[k]$ → IFFT

B L_h

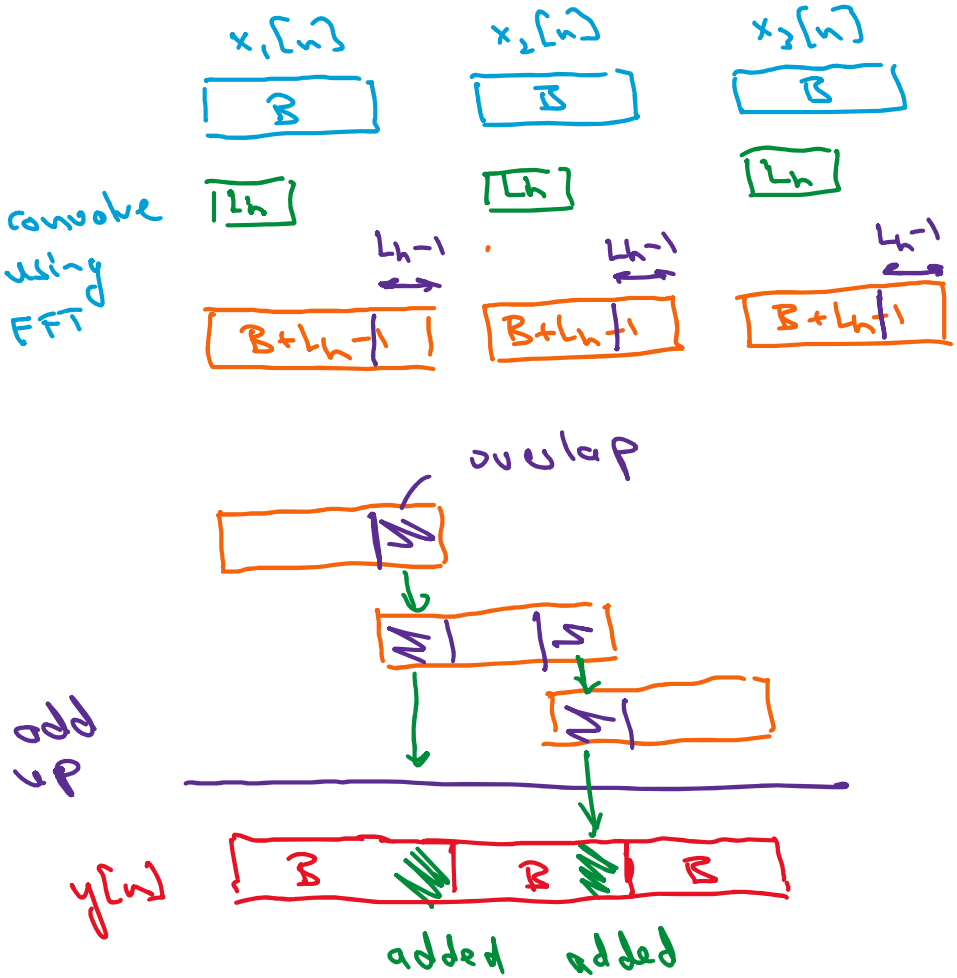
output of length $B+L_h-1$ for input of length B

Each block produces B + Lh − 1 valid linear-convolution samples.

take overlap of length L_h-1 at output and add that up

Why "add"?

Adjacent block convolutions overlap in time because the FIR has memory. Those overlapping contributions must be summed.



Overlap-Add Method

Tradeoff: Latency (NFFT) vs. computational efficiency ($O(NFFT \cdot \log NFFT)$)

output $y[n]$ has same length as input $x[n]$

there is latency between input and output because of FFT block processing
↳ $\approx$ NFFT

Overlap-Add Example

B=3, Lh=2, NFFT=B+Lh-1=4, h[n]=[h0, h1]

```
x[n]->   x0   x1   x2                   x3   x4   x5                   x6  x7  x8                       x9  ……

         *h[n]                          *h[n]                          *h[n]                          *h[n]

         x0h0 x0h1                      x3h0 x3h1                      x6h0 x6h1                      x9h0 ……
              x1h0 x1h1                      x4h0 x4h1                      x7h0 x7h1
                   x2h0 x2h1                      x5h0 x5h1                      x8h0 x8h1
         ---------------------------    ---------------------------    ---------------------------    -------
         x0h0 x0h1+x1h0 x1h1+x2h0 x2h1  x3h0 x3h1+x4h0 x4h1+x5h0 x5h1  x6h0 x6h1+x7h0 x7h1+x8h0 x8h1  x9h0 ……


overlap  x0h0 x0h1+x1h0 x1h1+x2h0 x2h1
and add                         x3h0 x3h1+x4h0 x4h1+x5h0 x5h1
                                                         x6h0 x6h1+x7h0 x7h1+x8h0 x8h1
                                                                                  x9h0 ……

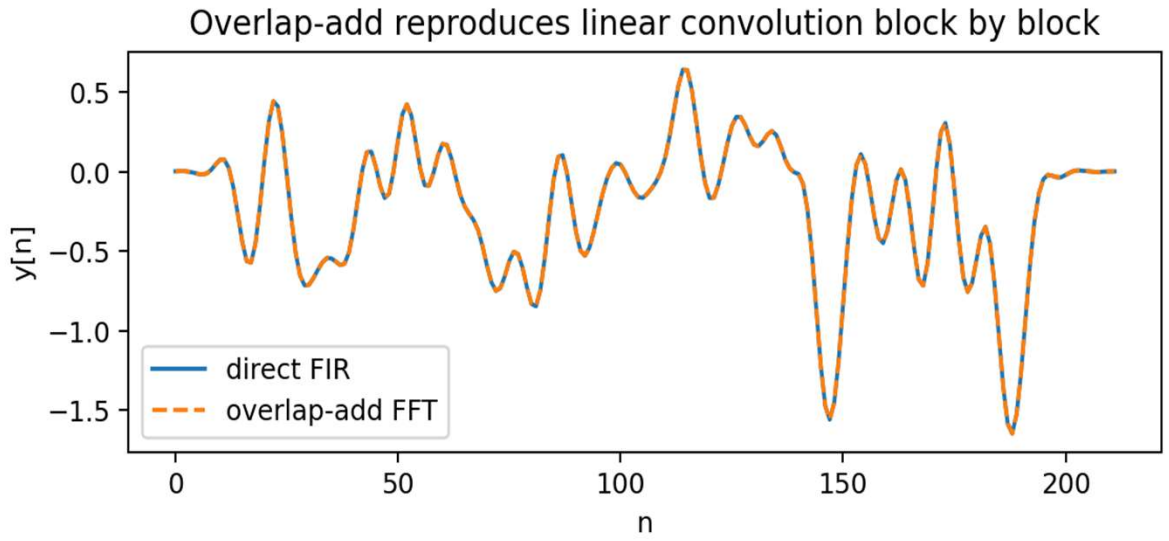
y[n]->   x0h0 x0h1+x1h0 x1h1+x2h0 x2h1+x3h0 x3h1+x4h0 x4h1+x5h0 x5h1+x6h0 x6h1+x7h0 x7h1+x8h0 x8h1+x9h0 ……
```

$$y[n] = \sum_{k=0}^{1} h[k]x[n-k] = \sum_{m=n-1}^{n} x[m]h[n-m]$$

Overlap-Add gives the same FIR output

OVERLAP-ADD

0:54–0:58



Verification

The direct time-domain FIR and FFT overlap-add traces lie on top of one another to numerical precision.

This equivalence should be a coding unit test.

Overlap-Save: overlap the input, discard invalid outputs

OVERLAP-SAVE need $NFFT \geq L_x + L_h + 1$ 0:58–1:06

- Choose an FFT size NFFT and FIR length Lh.
- Each input block contains Lh − 1 old samples plus new samples.
- Perform NFFT-point circular convolution.
- Discard the first Lh − 1 output samples, which contain time-aliasing contamination.
- Keep the remaining NFFT − Lh + 1 valid samples.

Why “save”?

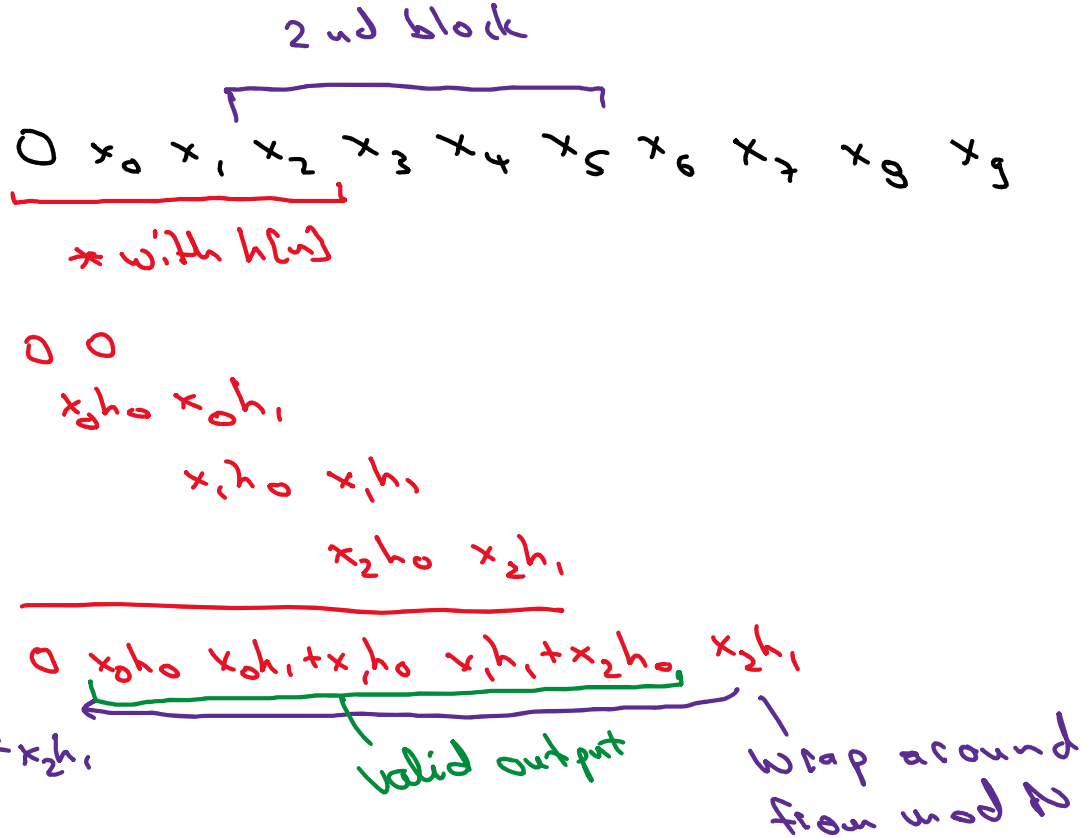
We save only the uncontaminated portion of each circular-convolution block and concatenate those portions.

Contrast

OLA overlaps and adds outputs. OLS overlaps inputs and discards corrupted outputs.

Example OLS

$L_x = 3 \quad L_h = 2 \quad L_y = 4 \quad N = 4 = 3+2-1 \quad h[n] = h_0, h_1$



Overlap-Save Example

B=3, Lh=2, NFFT=B+Lh-1=4, h[n]=[h0, h1]

```
                          x2 overlap                                  x5 overlap
x[n]->   0   x0   x1   x2                  x2   x3   x4   x5                        x5   x6   x7   x8

         *h[n]                             *h[n]                                   *h[n]

         0h0 0h1                           x2h0 x2h1                               x5h0 x5h1
             x0h0 x0h1                          x3h0 x3h1                               x6h0 x6h1
                  x1h0 x1h1                          x4h0 x4h1                               x7h0 x7h1
                       x2h0 x2h1                          x5h0 x5h1                               x8h0 x8h1
         ------------------------------    ---------------------------------------    ---------------------------------------
         0 x0h0 x0h1+x1h0 x1h1+x2h0 x2h1   x2h0 x2h1+x3h0 x3h1+x4h0 x4h1+x5h0 x5h1   x5h0 x5h1+x6h0 x6h1+x7h0 x7h1+x8h0 x8h1

wrap     0+x2h1 x0h0 x0h1+x1h0 x1h1+x2h0   x2h0+x5h1 x2h1+x3h0 x3h1+x4h0 x4h1+x5h0   x5h0+x8h1 x5h1+x6h0 x6h1+x7h0 x7h1+x8h0
mod NFFT

Discard and concatenate

y[n]->   x0h0 x0h1+x1h0 x1h1+x2h0 x2h1+x3h0 x3h1+x4h0 x4h1+x5h0 x5h1+x6h0 x6h1+x7h0 x7h1+x8h0 ……
```

$$y[n] = \sum_{k=0}^{1} h[k]x[n-k] = \sum_{m=n-1}^{n} x[m]h[n-m]$$

OLA versus OLS: engineering trade-offs

STREAMING

1:06–1:10

Overlap-Add

- Simple conceptual decomposition.
- Needs an output accumulation buffer.
- Input blocks are non-overlapping.

Overlap-Save

- Input blocks overlap by Lh − 1.
- No output addition step.
- Discard first Lh − 1 samples per block.

Both

Reuse H[k] for every block and can be much faster than direct convolution for long FIR filters.

Choosing FFT and block sizes

STREAMING

1:10–1:13

- Larger FFTs often improve arithmetic efficiency per sample but increase buffering latency.
- Short blocks reduce latency but increase FFT overhead.
- Choose NFFT using measured platform performance, not only asymptotic complexity.
- On an edge device, memory movement and cache behavior can matter as much as multiply counts.

Design question

A classifier needs 10-ms latency. Would a huge “efficient” FFT block still be acceptable? No—the end-to-end latency constraint dominates.

ML pipeline connection

ML BRIDGE

1:13–1:14

stream → fast FIR / resampling → framing → features → model

- Fast convolution lets expensive preprocessing run continuously without changing the filter response.
- The same block boundaries should not create artifacts that leak into ML features.
- Streaming DSP and streaming inference must share latency and buffer budgets.

Think like a system designer

Accuracy, throughput, latency, and numerical equivalence all matter.

Lecture 11 wrap-up

WRAP-UP

1:14–1:15

- The DFT samples the finite-record DTFT.
- DFT multiplication gives circular convolution.
- Zero-padding prevents wrap-around and recovers linear convolution.
- Overlap-add and overlap-save extend FFT convolution to endless streams.
- Block size is both a DSP and systems-design parameter.

Exit ticket

Why is NFFT ≥ B + Lh − 1 required in overlap-add, and what fails if NFFT is smaller?

One feature choice when using DFT/FFT is length of x[n] sequence L_x versus blocklength N_{FFT} of DFT/FFT

↑ should break up into small factors for efficiency of FFT

if N = 137 prime, for example then no advantage using FFT over DFT ⇒ $O(N^2)$

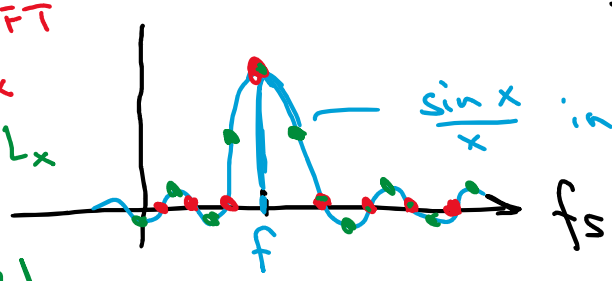
Two Reasons to Consider Zero Padding for DFT/FFTs

To go from L_x prime numbers to efficient FFT, use N_{FFT} with small prime factors, e.g. $N = 2^b$ and pad $x[n]$ with $N_{FFT} - L_x$ zeros

DFT/FFT

$N_{FFT} = L_x$

if $N_{FFT} = 2 \cdot L_x$ and $x[n]$ is zero padded



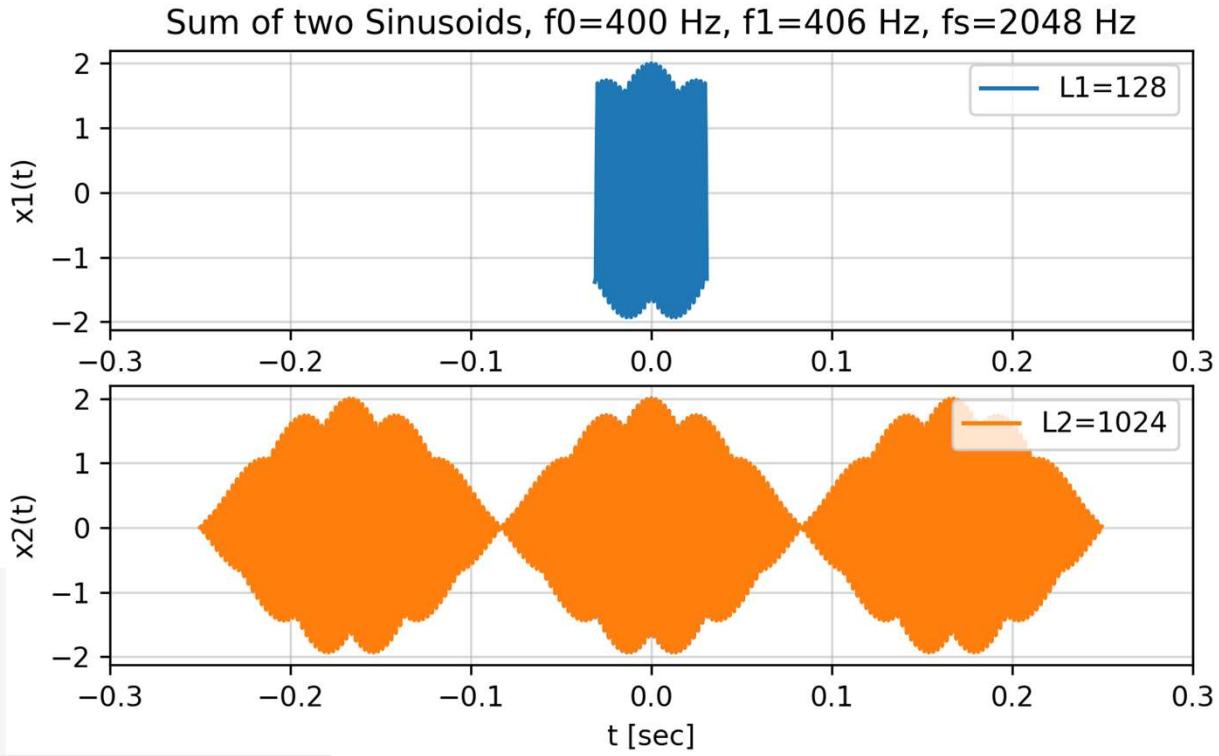
$\frac{\sin x}{x}$ in frequency because of truncation to length L_x

$x[n] = e^{j 2\pi f n / f_s}$

If we have two tones (sinusoids) with frequencies close together

Can we increase the frequency resolution between the two sinusoids by using zero padding?

Two Sinusoids Close in Frequency




```
# Parameters
fs = 2048    # sampling rate, samples/sec
L1, L2 = 128, 1024    # signal length in samples
f0, f1 = 400, 406     # frequencies in Hz
N1, N2 = 128, 4096    # FFT blocklengths
```

```
# Waveforms
tt1 = (np.arange(L1)-L1/2)/fs    # time axis 1, seconds
tt2 = (np.arange(L2)-L2/2)/fs    # time axis 2, seconds
x1 = np.exp(2j*np.pi*f0*tt1) + np.exp(2j*np.pi*f1*tt1)  # signal 1, length N1
x2 = np.exp(2j*np.pi*f0*tt2) + np.exp(2j*np.pi*f1*tt2)  # signal 1, length N2
```

Two Sinusoids Close in Frequency

```
# DFT/FFTs
X1_N1 = np.fft.fft(x1, n=N1)/L1
ff1 = np.fft.fftfreq(N1, 1/fs)
X1_N2 = np.fft.fft(x1, n=N2)/L1
ff2 = np.fft.fftfreq(N2, 1/fs)
X2_N2 = np.fft.fft(x2, n=N2)/L2
```

Blue to orange: Zero padding increases graph resolution but not frequency resolution.

Orange to green: More signal samples increases frequency resolution.

